

Name : N.Rohith 2520090191

Practical : WEEK 8

Task -1:

1.Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Code:

```
File: 72.c Modified
/* calloc() */
int *arr2 = (int *)calloc(5, sizeof(int));
if (arr2 == NULL)
{
    printf("calloc() failed.\n");
    free(arr1);
    return 1;
}

printf("calloc(): Allocated memory for 5 integers.\n");
printf("Initial calloc() values: ");
for (i = 0; i < 5; i++)
{
    printf("%d ", arr2[i]);
}
printf("\n");

/* realloc() */
int *temp = (int *)realloc(arr1, 10 * sizeof(int));
if (temp == NULL)
{
    printf("realloc() failed.\n");
    free(arr1);
    free(arr2);
    return 1;
}

arr1 = temp;
printf("realloc(): Resized first array from 5 to 10 integers.\n");
for (i = 5; i < 10; i++)
{
    arr1[i] = (i + 1) * 10;
}

printf("Values after realloc(): ");
for (i = 0; i < 10; i++)
{
    printf("%d ", arr1[i]);
}
printf("\n");

/* free() */
free(arr1);
arr1 = NULL;

free(arr2);
arr2 = NULL;

printf("free(): All dynamically allocated memory released.\n");

return 0;
}

Get Help      WriteOut      Read File      Prev Pg      Cut Text      Cur Pos
Exit          Justify       Where is       Next Pg      UnCut Text    To Spell
```

Output:

```
nrk@nallas-mac-Book-Air ~ % nano 72.c
nrk@nallas-mac-Book-Air ~ % gcc 72.c -o 72
nrk@nallas-mac-Book-Air ~ % ./72
malloc(): Allocated memory for 5 integers.
Values using malloc(): 10 20 30 40 50
calloc(): Allocated memory for 5 integers.
Initial calloc() values: 0 0 0 0 0
realloc(): Resized first array from 5 to 10 integers.
Values after realloc(): 10 20 30 40 50 60 70 80 90 100
free(): All dynamically allocated memory released.
nrk@nallas-mac-Book-Air ~ %
```

2. Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.

Code:

```

LM PID0 5.09                                     File: 75.c                                     Modified
printf("\nBefore fork()\n");
printf("Parent memory address: %p\n", (void *)memory);

pid = fork();

if (pid < 0)
{
    perror("fork");
    free(memory);
    return 1;
}

if (pid == 0)
{
    /* Child process */
    printf("\nChild process created.\n");
    printf("Child PID: %d\n", getpid());
    printf("Child memory address: %p\n", (void *)memory);

    printf("\nChild is modifying 10 MB...\n");

    /* Modify the memory */
    for (size_t i = 0; i < SIZE; i++)
    {
        memory[i] = 'B';
    }

    printf("Child modification completed.\n");
    printf("Child memory address after modification: %p\n",
        (void *)memory);

    sleep(10);

    free(memory);
    exit(0);
}
else
{
    /* Parent process */
    printf("\nParent continues running.\n");
    printf("Parent PID: %d\n", getpid());

    sleep(5);

    printf("\nParent memory is still unchanged.\n");
    printf("First byte in parent: %c\n", memory[0]);

    wait(NULL);

    free(memory);
}

return 0;
}

```

Output:

```

nrk@nallas-mac-Book-Air ~ % nano 75.c
nrk@nallas-mac-Book-Air ~ % gcc 75.c -o 75
nrk@nallas-mac-Book-Air ~ % ./75
Parent PID: 1626
Allocated and initialized 10 MB.

Before fork()
Parent memory address: 0x47a00000

Parent continues running.
Parent PID: 1626

Child process created.
Child PID: 1627
Child memory address: 0x47a00000

Child is modifying 10 MB...
Child modification completed.
Child memory address after modification: 0x47a00000

Parent memory is still unchanged.
First byte in parent: A
nrk@nallas-mac-Book-Air ~ %

```

